

ShelfSight

AI-Assisted Library Management System

Team Name: Team 10

Supervisor: Professor Sridhar Alagar

Project Coordinator: Adarsh Gella

Institution: Erik Jonsson School of Engineering and Computer Science, UT Dallas

Date: May 2026

Table of Contents

- Executive Summary
- 1. Introduction
- 2. Project Specification & Management
- 3. System Architecture & High-Level Design
- 4. Implementation & AI Orchestration
- 5. System Performance & Validation
- 6. Lessons Learned
- 7. Conclusion & Future Work
- References
- Acknowledgements

Executive Summary

ShelfSight is an AI-assisted library management system designed to modernize how physical libraries catalog, organize, and manage inventory. The system integrates a full-stack web application with AI-powered ingestion, enabling librarians to upload book images and automatically extract metadata, detect ISBNs, and receive Dewey Decimal classification suggestions.

The final system delivers a complete end-to-end workflow covering authentication, catalog management, circulation, ingestion, and a spatial library map interface. The platform was validated through automated testing, end-to-end testing, and multi-user performance testing.

Key results include:

- **53 automated tests** with **0 failures and 0 flaky tests**
- **30 end-to-end tests passed** using Playwright
- **9.4k+ requests handled under load** with **0% failure rate**
- Stable performance at **45 concurrent users**, with scalability tested up to **135 concurrent users**

The system is currently at a **pre-launch release candidate stage**, fully functional and ready for demonstration, with documented limitations and future scaling considerations.

1. Introduction

1.1 Motivation & Problem Statement

Traditional library management systems rely heavily on manual data entry, disconnected catalog systems, and inefficient physical navigation. Librarians must manually input book metadata, organize shelves without intelligent support, and rely on non-visual catalog systems that do not reflect real-world layouts.

This creates inefficiencies in:

- Cataloging new books
- Locating books physically
- Managing large-scale collections
- Maintaining accurate metadata

ShelfSight addresses these challenges by combining automation, AI, and visualization into a unified system.

1.2 Objectives, Scope, and Goals Achieved

Objectives

- Automate cataloging using AI
- Provide real-time catalog management
- Connect digital records to physical shelf locations
- Support scalable library operations

Scope (MVP)

- Authentication and role-based access
- Catalog CRUD operations
- Circulation system (checkout/check-in)
- AI ingestion pipeline
- Interactive 2D library map

Goals Achieved

- Fully functional full-stack system
- AI-assisted ingestion workflow
- End-to-end validated workflows
- Load-tested backend performance
- CI/CD and deployment readiness

1.3 Proposed Solution

ShelfSight provides an integrated platform where:

- Users upload book images
- AI extracts metadata and classification
- Data is validated and stored
- Books are mapped to physical shelves
- Users interact with a visual library interface

This reduces manual effort while improving accuracy and usability.

1.4 Key Highlights

- AI-powered ingestion using OCR + metadata enrichment
- Visual 2D map connecting catalog to physical layout
- Full CI/CD pipeline with automated validation
- Scalable backend tested under concurrent load
- Strong verification: zero failures across all automated tests

2. Project Specification & Management

2.1 Project Specification (Single Source of Truth)

The system was built using a detailed technical specification that defined:

- API contracts
- database schema
- system architecture
- strict technology constraints

This document served as the **instruction set for AI-assisted development**

2.2 Maintaining the Single Source of Truth

- GitHub used for version control
- Spec updated alongside development
- Changes validated before implementation
- Ensured AI-generated code remained aligned

2.3 Key Milestones

- Initial system architecture completed
- Authentication + API layer implemented
- Catalog + circulation fully functional
- AI ingestion integrated
- E2E testing completed
- CI/CD + release readiness finalized

2.4 Team Roles & Responsibilities

Each team member contributed to:

- Backend services
- Frontend interfaces
- Testing and validation
- Performance optimization
- Documentation and CI/CD

All contributions were consolidated into a final release-ready system.

3. System Architecture & High-Level Design

3.1 System Overview

ShelfSight follows a **three-layer architecture**:

- Frontend: Next.js
- Backend: Node.js + Express
- Database: PostgreSQL with Prisma ORM
- AI Layer: AWS Textract + OpenAI

3.2 Architecture Design

Key components:

- REST API endpoints for all operations
- Modular services for business logic
- AI ingestion pipeline for automation
- Interactive UI for user workflows

3.3 Key Architectural Decisions

- Prisma ORM for maintainability
- PostgreSQL for scalability
- JWT authentication for security
- AI-assisted ingestion instead of manual entry

Trade-offs:

- Synchronous ingestion used for MVP
- Async pipeline planned for future scaling

4. Implementation & AI Orchestration

4.1 Technologies Used

- Frontend: Next.js, React, Tailwind
- Backend: Node.js, Express
- Database: PostgreSQL
- ORM: Prisma
- AI: OpenAI + AWS Textract
- Testing: Playwright, Vitest, k6

4.2 AI Build Strategy

AI was used to:

- Generate code based on specification
- Assist with backend logic
- Improve ingestion pipeline
- Optimize development speed

4.3 Human-in-the-Loop

Manual intervention was required for:

- Debugging AI-generated code
- Fixing ingestion reliability issues
- Resolving validation and API inconsistencies
- Performance tuning

4.4 Codebase

Full project hosted on GitHub with:

- frontend repo
- backend repo
- CI/CD workflows
- testing pipelines

5. System Performance & Validation

5.1 Testing Strategy

- Unit + integration tests (Vitest)
- End-to-end testing (Playwright)
- Load testing (k6)

5.2 Technical Performance Metrics

Automated Testing

- 53 total tests
- 51 passed
- 0 failures

Load Testing

- 9,453 HTTP requests
- 45 concurrent users
- 0% failure rate

5.3 Post-Scaling Performance

After scaling to **10,000+ books**:

- p95 latency increased significantly under load
- System remained **100% stable (0 failures)**

Key results:

- 45 VUs → ~3152 ms p95
- 90 VUs → ~7974 ms p95
- 135 VUs → ~16019 ms p95

Conclusion:

- System is **functionally stable at scale**
- Performance optimization needed for high concurrency

5.4 AI Performance

- ISBN detection and metadata extraction successful
- Dewey classification provided via AI or fallback heuristic
- System maintains reliability even without API keys

6. Lessons Learned

6.1 Insights

- AI accelerates development but requires strong constraints
- Clear specifications are critical for success
- Testing is essential for validating AI-generated systems

6.2 Challenges

- Handling large-scale data ingestion
- Performance degradation under load
- Debugging AI-generated inconsistencies

6.3 Skills Developed

- AI-assisted engineering
- Full-stack system design
- Performance testing and scaling
- CI/CD and deployment readiness

6.4 Reflection

The project demonstrated how AI can augment development workflows while requiring structured oversight and validation.

7. Conclusion & Future Work

7.1 Key Accomplishments

- Fully functional library management system

- AI-powered ingestion workflow
- Verified system reliability and stability
- Deployment-ready architecture

7.2 Future Enhancements

- Multi-tenancy support
- Full async ingestion pipeline
- Advanced analytics dashboard
- Improved performance optimization
- Large-scale dataset validation (100k+)

References

- [AWS Textract Documentation](#)
- [OpenAI API Documentation](#)
- [Prisma ORM Documentation](#)
- [Next.js Documentation](#)
- [k6 Load Testing Documentation](#)

Acknowledgements

We would like to thank:

- Professor Sridhar Alagar
- Adarsh Gella
- Course staff and mentors

for their guidance and support throughout the project.